OOAD Project Specification Document

Bhavith Kumar Yarava (PES1UG23CS146)

B A Saharsh (PES1UG23CS130)

Dhanush Rao K (PES1UG23CS182)

Bhumireddy Sai Dhanush Reddy (PES1UG23CS148)

Project Title: Student Learning Management System (LMS)

---

1. Introduction

The Student Learning Management System (LMS) is designed to digitize and streamline academic activities between students, instructors, and administrators. The system centralizes course management, assignment workflows, performance tracking, and communication between stakeholders. This project aims to model the LMS using Object-Oriented Analysis and Design (OOAD) principles, including use case modeling, class modeling, interaction diagrams, and system architecture.

---

2. Objectives

- To create a scalable digital platform supporting student learning and instructor content delivery.
- To automate assignment submission, grading, and progress tracking.
- To enable administrators to efficiently manage users and course data.
- To design a structured OOAD model that aligns with real-world system behaviour.

---

3. Scope of the System

The LMS will include functionalities for:

Students

- Register/login
- Enroll in available courses
- Access course materials (videos, PDFs, slides)
- Submit assignments
- View grades
- Track course progress

- Receive deadline notifications

Instructors

- Create and manage course modules
- Upload study materials (PDF, videos)
- Create assignments and set deadlines
- View student submissions
- Grade and give feedback

Administrators

- Create and manage user accounts
- Add/remove courses
- Assign instructors to courses
- Generate performance summaries and reports

---

4. System Requirements

4.1 Functional Requirements

FR1: User Authentication

- The system shall allow users (students, instructors, administrators) to register and login.

FR2: User Role Management

- The system shall assign permissions based on role.

FR3: Course Management

- Admin can create, update, delete courses.
- Instructor can create modules within the course.
- Students can enroll in courses.

FR4: Material Management

- Instructors can upload course content.
- Students can download/view materials.

FR5: Assignment Management

- Instructor can create assignments with deadlines.
- Students can submit assignments digitally.
- Instructor can grade submissions.

FR6: Progress Tracker

- The system shall automatically compute course completion percentage.
- The system shall notify students of upcoming deadlines.

FR7: Report Generation

- Admin can generate student performance reports.

4.2 Non-Functional Requirements

- **Usability:** Intuitive UI, accessible on desktop/mobile.
- **Performance:** System should support 1000+ concurrent users.
- **Security:** Secure authentication, encrypted data storage.
- **Reliability:** 99% uptime.
- **Scalability:** Easy addition of new courses/users/modules.

---

5. Actors

Actor	Description
Student	End-user who consumes course content
Instructor	Subject expert who creates content and grades assignments
Administrator	System manager who handles user and course operations
Notification Service (External System)	Sends reminders to students

---

6. Use Case List

Student Use Cases

1. Register/Login
2. Enroll in Course
3. View Course Materials
4. Submit Assignment
5. View Grades
6. Track Progress
7. Receive Notifications

Instructor Use Cases

1. Create Course Modules

2. Upload Materials
3. Create Assignments
4. Grade Submissions
5. View Student Progress

Administrator Use Cases

1. Manage User Accounts
2. Manage Courses
3. Assign Instructors
4. Generate Reports

---

7. Detailed Use Case Specifications

Use Case 1: Enroll in Course (Student)

- **Actor:** Student
- **Precondition:** Student must be logged in.
- **Main Flow:**
    1. Student views available courses.
    2. Selects a course to enroll.
    3. System adds the course to the student's dashboard.
- **Postcondition:** Enrollment reflected in student profile.
- **Exception:** Course is full or not available.

Use Case 2: Create Assignment (Instructor)

- **Actor:** Instructor
- **Precondition:** Instructor must be assigned to the course.
- **Main Flow:**
    1. Instructor creates assignment and sets deadline.
    2. System publishes assignment to enrolled students.
- **Postcondition:** Assignment visible to all students.
- **Exceptions:** Invalid input, missing materials.

---

8. Class Diagram (Specification Level)

Classes

- **User**
  - Attributes: userId, name, email, password, role
  - Methods: login(), updateProfile()
- **Student** (inherits User)
  - Attributes: enrolledCourses[], submissions[]
  - Methods: enrollCourse(), submitAssignment(), viewGrades(), viewProgress()
- **Instructor** (inherits User)
  - Attributes: coursesHandled[]
  - Methods: createModule(), uploadMaterial(), createAssignment(), gradeSubmission()
- **Administrator** (inherits User)
  - Methods: manageUser(), manageCourse(), assignInstructor(), generateReport()
- **Course**
  - Attributes: courseId, title, description, modules[], assignments[]
  - Methods: addModule(), removeModule()
- **Module**
  - Attributes: moduleId, title, materials[]
  - Methods: addMaterial()
- **Material**
  - Attributes: fileId, fileType, uploadDate
  - Methods: getContent()
- **Assignment**
  - Attributes: assignmentId, description, deadline, submissions[]
  - Methods: publish(), evaluate()
- **Submission**
  - Attributes: submissionId, studentId, timestamp, grade
  - Methods: submit(), updateGrade()
- **ProgressTracker**
  - Attributes: totalModules, completedModules
  - Methods: calculateProgress(), notifyDeadlines()

---

9. Sequence Diagrams to be Included

(You will create these later)

- Student enrolling in a course
- Student submitting assignment
- Instructor grading assignment
- System sending deadline notifications

---

10. System Architecture (High Level)
• **Presentation Layer:** Java Spring Boot MVC for user interface, controllers, and request handling.
• **Application Layer:** Core LMS business logic including course management, progress tracking, grading, and deadline notifications.
• **Database Layer:** SQL database (MySQL/PostgreSQL) storing users, courses, materials, assignments, and submissions.

---

11. Constraints

- Users must have unique IDs
- Materials must follow size and format restrictions
- Progress tracker must auto-update after each module completion

---

12. Future Enhancements

- Integration with video conferencing platforms
- Adaptive learning paths